

ArtSea

Scrum: Retrospektywa Sprintu

Benjamin Jurewicz
198326

Piotr Kierznowski
197652

Marta Kociszewska
198143

Lidia Zawrzykraj
198257

1. O PROJEKCIE I PRODUKCIE

Nazwa i Ogólna Koncepcja

- **ArtSea** - system zarządzania portfolio artysty
- **Ogólna koncepcja** - system planowany jest jako platforma, umożliwiająca artystom błyskawiczne generowanie własnych stron internetowych z ich twórczością

Adresowany Problem i Obszar Zastosowania

- **Problemy:**
 - Trudność techniczna: Artyści nie potrafią samodzielnie postawić (stworzyć) strony
 - Brak spójności: Portfolio w mediach społecznościowych są rozproszone i niespójne
 - Wysokie koszty: Zatrudnienie dewelopera jest zbyt drogie dla początkujących twórców
- **Obszar zastosowania:**
 - Sektor kreatywny, promocja sztuki w Internecie oraz digitalizacja dorobku artystycznego.

Rynek i Organizacja

- **Skala działalności:**
 - Projekt adresowany jest do twórców na poziomie krajowym i europejskim.
- **Rynek:**
 - Artyści niezależni, studenci uczelni artystycznych oraz małe galerie sztuki.

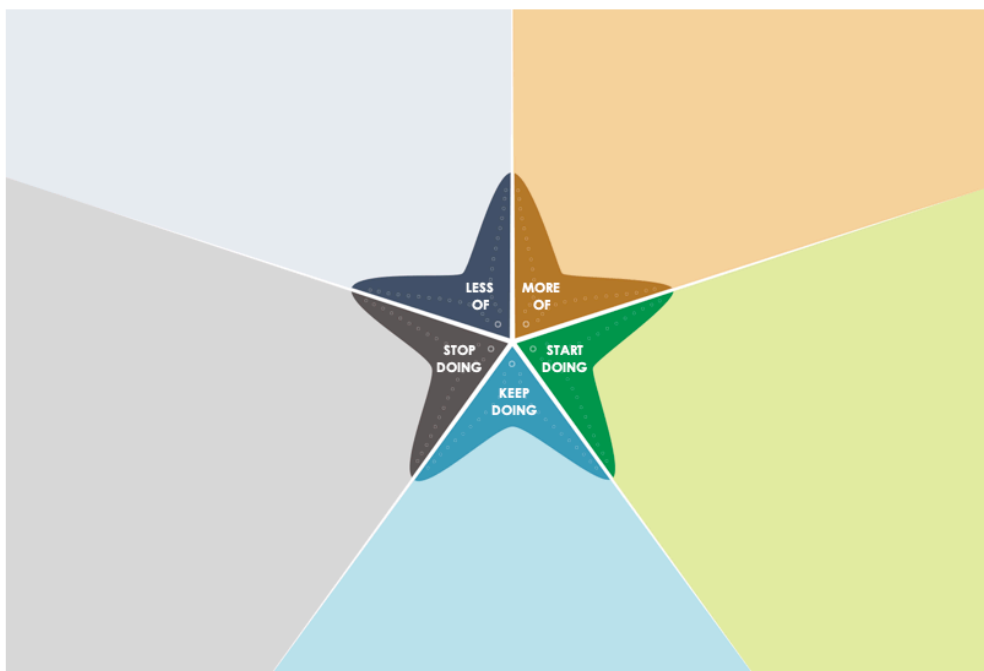
2. WYBRANA TECHNIKA RETROSPEKTYWY

Do przeprowadzenia retrospektywy pierwszego sprintu wybrano technikę **Starfish Retrospective** (retrospektywa „rozgałęziona”). Jest to metoda pracy z tablicą podzieloną na pięć pól: **Keep, Less, More, Start** i **Stop**. Uczestnicy, na karteczkach, zapisują obserwacje z ostatniej iteracji w odpowiednich kategoriach, następnie grupują podobne wpisy, omawiają najważniejsze tematy i wybierają niewielką liczbę działań usprawniających na kolejny sprint.

Kategorie zastosowane podczas spotkania oznaczały:

- **Keep** – praktyki, które sprawdziły się w sprincie i powinny być utrzymane.
- **Less** – działania potrzebne, ale wykonywane zbyt często lub zbyt szczegółowo.
- **More** – praktyki wartościowe, którym należy poświęcić więcej uwagi.
- **Start** – nowe działania, których brak był widoczny w zakończonej iteracji.
- **Stop** – zachowania i elementy procesu, które spowalniają zespół i powinny zostać ograniczone lub usunięte.

STARFISH RETROSPECTIVE



Rysunek 1: Schemat techniki Starfish użytej podczas retrospektywy

Źródła: [EasyRetro – Starfish Retrospective](#), [Atlassian Team Playbook – Retrospectives](#)

Uzasadnienie Wyboru

Technika Starfish dobrze pasowała do projektu ArtSea z kilku powodów. Po pierwsze, pierwszy sprint obejmował równolegle różne typy prac: infrastrukturę techniczną, autoryzację, projekt bazy danych, konfigurację narzędzi, makiety i design system. Prosty podział na „co poszło dobrze” i „co poszło źle” byłby zbyt ogólny, ponieważ zespół potrzebował rozróżnić praktyki do utrzymania, ograniczenia, wzmocnienia oraz rozpoczęcia.

Po drugie, zespół jest niewielki i pracuje hybrydowo, częściowo zdalnie. Starfish jest lekki organizacyjnie, nie wymaga długiego szkolenia i można go łatwo przeprowadzić na wirtualnej tablicy. Każdy uczestnik może najpierw samodzielnie dopisać obserwacje, a dopiero później omówić je z resztą zespołu. Zmniejsza to ryzyko dominacji jednej osoby w dyskusji.

Po trzecie, metoda wspiera wybór konkretnych działań poprawy. Kategorie **Start**, **More**, **Less** i **Stop** naturalnie prowadzą do sformułowania action items, a kategoria **Keep** pozwala docenić elementy procesu, które już działają.

3. PRZEBIEG I WYNIKI RETROSPEKTYWY

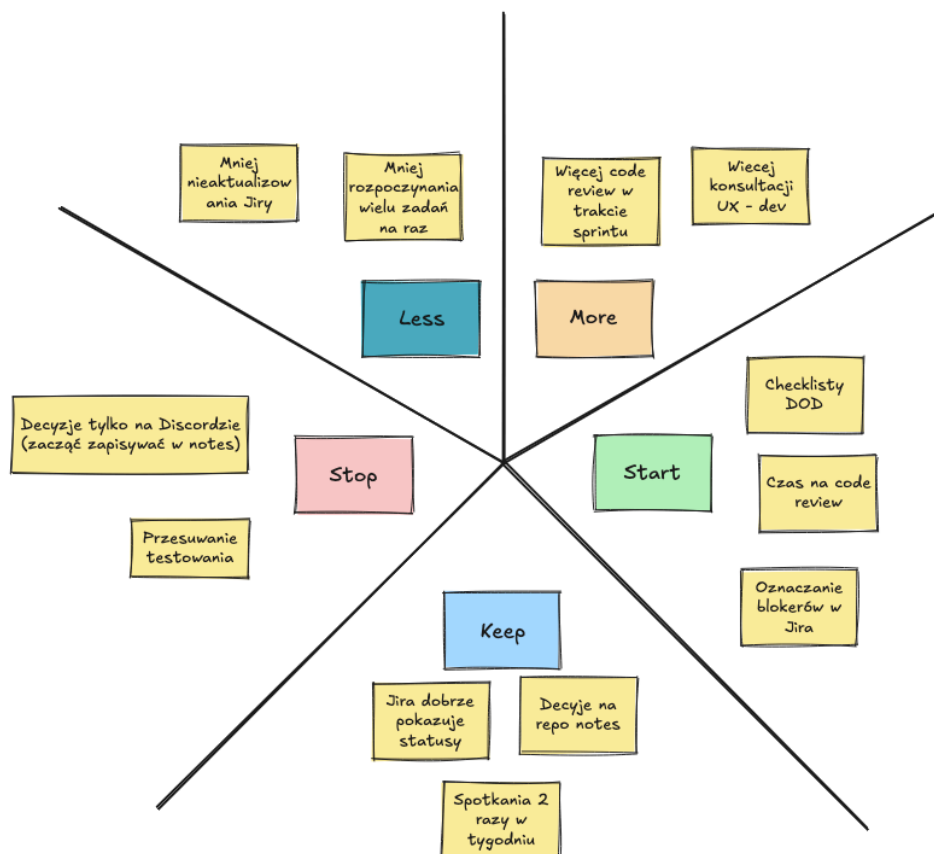
Retrospektywa dotyczyła pierwszego dwutygodniowego sprintu, którego celem było uruchomienie fundamentów projektu: konfiguracji środowiska, narzędzi deweloperskich, autoryzacji, projektu bazy danych oraz podstaw projektowych w Figmie. Spotkanie odbyło się zdalnie na platformie Discord. Do pracy wykorzystano wirtualną tablicę z układem Starfish oraz tablicę Jira jako źródło informacji o statusach zadań.

Tabela 1: Organizacja retrospektywy

Element	Opis
Forma spotkania	Zdalnie, rozmowa głosowa na Discordzie oraz wspólna tablica online
Czas trwania	Około 75 minut, czyli poniżej timeboxu 1,5 h dla dwutygodniowego sprintu
Uczestnicy	Benjamin Jurewicz, Marta Kociszewska, Lidia Zawrzykraj, Piotr Kierznowski
Moderator	Piotr Kierznowski
Osoba notująca	Marta Kociszewska
Narzędzia	Discord, Jira, wirtualna tablica retrospektywy
Dane wejściowe	Backlog sprintu, tablica Kanban, Definition of Done, notatki zespołu i obserwacje uczestników

Spotkanie przeprowadzono w następujących krokach:

1. **Wprowadzenie i przypomnienie celu** – moderator przypomniał, że celem spotkania jest poprawa procesu pracy, a nie wskazywanie winnych. Ustalono, że zespół skupia się na ostatnim sprincie.
2. **Cicha praca indywidualna** – każdy uczestnik przez kilka minut dopisywał obserwacje do pięciu pól rozwiązydy.
3. **Grupowanie wpisów** – podobne notatki połączono w tematy, m.in. komunikację, infrastrukturę, projektowanie UI, code review i aktualność zadań w Jira.
4. **Dyskusja** – zespół omówił najważniejsze grupy, szukając przyczyn oraz możliwych usprawnień.
5. **Selekcja działań** – propozycje usprawnień poddano głosowaniu kropkami. Każdy uczestnik miał trzy głosy i mógł rozdzielić je między tematy, które uważał za najbardziej wpływowe dla kolejnego sprintu.
6. **Podsumowanie** – wybrano trzy action items oraz określono oczekiwany efekt każdego z nich.



Rysunek 2: Tablica retrospektywy po zakończeniu pracy zespołu

Wyniki w Kategoriach Starfish

Najważniejsze obserwacje zespołu zostały uporządkowane zgodnie z pięcioma kategoriami techniki Starfish:

- **Keep** – utrzymać praktyki, które sprawdziły się w sprincie:
 - pracę z tablicą Jira i jawne statusy zadań,
 - krótkie spotkania statusowe dwa razy w tygodniu,
 - dokumentowanie decyzji w repozytorium notes.
- **Less** – ograniczyć działania, które spowalniały pracę lub były wykonywane w nieoptymalny sposób:
 - doprecyzowywanie rozwiązań technicznych bez wcześniejszego sprawdzenia ich wykonalności,
 - odkładanie aktualizacji statusów w Jira,
 - równoległe rozpoczynanie nowych zadań, gdy istnieją zadania czekające na review lub testy.
- **More** – robić częściej praktyki, które poprawiają jakość i przewidywalność pracy:
 - krótkie przeglądy pull requestów w trakcie sprintu,
 - konsultacje między UX a implementacją przed rozpoczęciem większych zadań,
 - wczesne testowanie lokalnej konfiguracji przez osoby inne niż autor zmiany.
- **Start** – rozpocząć nowe działania usprawniające proces:
 - stosowanie checklisty Definition of Done przy zamykaniu zadań,
 - oznaczanie blokerów bezpośrednio w Jira,
 - przygotowywanie krótkich notatek z decyzji projektowych po spotkaniach,
 - planowanie slotów na code review, aby nie kumulowały się pod koniec sprintu.
- **Stop** – przestać stosować praktyki, które generują niejasności lub opóźnienia:
 - traktowanie zadania jako prawie gotowe bez aktualnych kryteriów akceptacji,
 - zostawianie niedoprecyzowanych decyzji architektonicznych wyłącznie w rozmowach na Discordzie,
 - odraczanie testowania manualnego

Najważniejsze Sukcesy Sprintu

Za największy sukces uznano stworzenie podstaw technicznych i organizacyjnych projektu. Zespół doprecyzował backlog sprintu, opisał Definition of Done oraz rozpoczął prace nad infrastrukturą potrzebną do dalszej implementacji. Pozytywnie oceniono również równoległość prac: część zespołu mogła pracować nad architekturą i środowiskiem, a część nad makietami oraz design systemem.

Dobrze zadziały stałe spotkania statusowe oraz centralne przechowywanie dokumentacji w repozytorium [notes](#). Dzięki temu decyzje dotyczące produktu, technologii i procesu nie były rozproszone między prywatnymi wiadomościami. Uczestnicy docenili też, że Jira pozwalała szybko zobaczyć, które zadania są w toku, które czekają na review, a które są zablokowane.

Najważniejsze Problemy i Ryzyka

Najczęściej powtarzającym się problemem była aktualność informacji o stanie zadań. Część statusów w Jira była zmieniana dopiero po wykonaniu większej porcji pracy, przez co obraz sprintu nie zawsze odpowiadał rzeczywistości. Drugim problemem było ryzyko wąskiego gardła w code review i testach. Ponieważ zespół jest mały, opóźnienie jednej osoby w przeglądzie zmian może zatrzymać kilka kolejnych zadań.

Zespół zwrócił też uwagę na potrzebę wcześniejszego uzgadniania styku UX i implementacji. Makiety oraz design system są kluczowe, ale jeżeli decyzje projektowe nie są od razu konfrontowane z ograniczeniami technicznymi, mogą powodować przeróbki w kolejnych sprintach. W obszarze dokumentacji problemem było to, że część decyzji była jasna dla uczestników rozmowy, ale nie zawsze od razu trafiała do trwałych notatek.

Propozycje Usprawnień Rozważane Przez Zespół

W dyskusji pojawiły się następujące propozycje: wprowadzenie checklisty zamknięcia zadania, rezerwacja czasu na code review, krótsze opisy decyzji w repozytorium [notes](#), aktualizowanie Jiry minimum raz dziennie, dopisywanie kryteriów akceptacji przed rozpoczęciem pracy oraz wcześniejsze konsultowanie większych elementów UI z osobą odpowiedzialną za implementację. Wszystkie propozycje zostały zapisane na tablicy, a następnie ocenione w głosowaniu kropkami.

4. ZADANIA DO WYKONANIA – ACTION ITEMS

Spośród zgłoszonych propozycji zespół wybrał trzy najważniejsze zadania poprawy. Zastosowano głosowanie kropkami: każdy uczestnik otrzymał trzy głosy, które mógł przydzielić do dowolnych propozycji. Wybrano te działania, które uzyskały najwięcej głosów, a jednocześnie były możliwe do wdrożenia już w kolejnym sprincie.

Tabela 2: Wybrane action items na kolejny sprint

Action item	Uzasadnienie	Kryterium wykonania
Codzienna aktualizacja statusów w Jira	Poprawi transparentność i ograniczy sytuacje, w których tablica nie pokazuje rzeczywistego postępu.	Każde aktywne zadanie ma aktualny status, osobę przypisaną i ewentualny bloker przed końcem dnia pracy.
Checklisty Definition of Done przed przeniesieniem do Done	Zmniejszy ryzyko zamykania zadań bez testów, review albo aktualnej dokumentacji.	Dla zadań kończonych w kolejnym sprincie autor potwierdza spełnienie checklisty DoD w komentarzu lub opisie zadania.
Stały slot na code review i testy manualne	Ograniczy kumulowanie review i testów pod koniec sprintu oraz pomoże utrzymać płynny przepływ przez kolumny IN REVIEW i IN TESTING.	W harmonogramie zespołu znajdują się co najmniej dwa krótkie sloty tygodniowo na review/testy, a zadania nie zalegają w review dłużej niż trzy dni robocze bez komentarza.

Dodatkowo ustalono, że mniej pilne propozycje, takie jak rozbudowanie szablonu notatek ze spotkań lub pełniejsza automatyzacja testów, zostaną ponownie ocenione podczas kolejnego planowania sprintu. Nie trafiły one do głównych action items, ponieważ w najbliższej iteracji ważniejsze jest ustabilizowanie podstawowego przepływu pracy.

5 . WNIOSKI

Technika Starfish okazała się odpowiednia dla zespołu ArtSea. Była łatwa do wyjaśnienia i nie wymagała dodatkowego przygotowania poza utworzeniem tablicy z pięcioma polami. Uczestnicy szybko zrozumieli różnice między kategoriami, a praca indywidualna na początku spotkania pomogła zebrać więcej obserwacji niż sama otwarta dyskusja.

Największą zaletą techniki było to, że nie prowadziła wyłącznie do listy problemów. Kategoria **Keep** pozwoliła nazwać praktyki, które warto utrzymać, natomiast **More, Less, Start** i **Stop** pomogły przekształcić obserwacje w konkretne decyzje procesowe. Dzięki temu retrospektywa zakończyła się trzema realistycznymi action items zamiast długą listą życzeń.

Ograniczeniem metody jest konieczność pilnowania czasu dyskusji. Przy pięciu kategoriach łatwo poświęcić zbyt dużo uwagi samej klasyfikacji karteczek, dlatego ważna była rola moderatora. Piotr pilnował przechodzenia między etapami spotkania i przypominał, że celem końcowym jest wybór działań poprawy.

Z perspektywy celów retrospektywy spotkanie spełniło swoje zadanie. Zespół omówił zarówno sukcesy, jak i problemy pierwszego sprintu, wskazał ryzyka dla dalszej pracy oraz wybrał działania możliwe do wdrożenia od razu w następnym sprincie. Satysfakcja uczestników była wysoka, ponieważ metoda była konkretna, angażująca i pozwoliła uporządkować dyskusję bez nadmiernej formalizacji.